

HUFFMAN CODING

Definition:

Huffman Coding is a lossless data compression technique that assigns shorter binary codes to more frequent symbols and longer codes to less frequent symbols, reducing the total number of bits needed to store or transmit data.

Or,

It's a smart way of replacing characters with bits so that the most common ones use fewer bits and rare ones use more.

Main Goal: Reduce the size of data without losing any information.

How? By assigning variable-length binary codes to symbols, where:

- High-frequency symbols → shorter codes.
- Low-frequency symbols → longer codes.

Importance:

- Saves storage space.
- Speeds up data transfer (less data to send).
- Forms the basis of many compression formats like ZIP, PNG, JPEG.
- Is optimal among all prefix-free coding methods.

Applications:

- File compression: ZIP, GZIP, TAR.
- Image compression: PNG uses Huffman coding.
- Multimedia compression: JPEG, MP3.
- Network transmission: Compress messages before sending.
- Text compression: Reducing storage of large text files.

How It Works (Step-by-step): (Algorithm Steps)

1. Count frequencies of each symbol in the input.
2. Create leaf nodes for each symbol with its frequency.
3. Build a priority queue/min-heap sorted by frequency.
4. Combine two smallest nodes into a new node whose frequency = sum of both.
5. Repeat step 4 until only one node (root) remains → Huffman Tree.
6. Assign codes: left branch = 0, right branch = 1.
7. Encode data using these codes.

Concept (Key Ideas):

- Variable-length codes (unlike fixed 8-bit ASCII).
- Prefix-free property: No code is the prefix of another code.

- Optimality: Produces the minimum possible average code length for given symbol frequencies.

Mathematical Derivation:

Given:

- Symbols: $s_1, s_2, \dots, s_n$
- Probabilities: $p_1, p_2, \dots, p_n$
- Code lengths: $l_1, l_2, \dots, l_n$

Average code length (L):
$$L = \sum_{i=1}^n p_i \cdot l_i$$

Goal: Minimize L subject to the Kraft-McMillan inequality:
$$\sum_{i=1}^n 2^{-l_i} \leq 1$$

Huffman algorithm constructs a tree that minimizes L.

Examples 1:

Text: "AABACD"

Symbol	Frequency	Probability	Code
A	3	$3/6 = 0.5$	0
B	1	$1/6 \approx 0.17$	10
C	1	$1/6 \approx 0.17$	110
D	1	$1/6 \approx 0.17$	111

Encoded: AABACD $\rightarrow$ 0 0 10 0 110 111 $\rightarrow$ **00100110111** (11 bits Vs. $6 \times 8 = 48$ bits in ASCII).

e.g.

ASCII Encoding:

- ASCII stores **every character** using **8 bits** (1 byte), no matter what the character is.
- Text is: A A B A C D
- Number of characters = 6
- Bits needed in ASCII = $6 \times 8 = 48$ bits

So, in ASCII, you would use:

A → 01000001 (8 bits)

B → 01000010 (8 bits)

C → 01000011 (8 bits)

D → 01000100 (8 bits)

Total: **48 bits**.

But, in Huffman Coding:

the actual bit count is **11** because:

3 A's → $3 \times 1 \text{ bit} = 3 \text{ bits}$

1 B → 2 bits

1 C → 3 bits

1 D → 3 bits

So, $3 + 2 + 3 + 3 = 11 \text{ bits}$.

Comparison ASCII Vs. Huffman:

- ASCII size = $6 \times 8 = 48 \text{ bits}$
- Huffman size = **11 bits**
- Savings: $\frac{48 - 11}{48} \times 100\% \approx 77.08\%$

Examples 2:

Symbols & frequencies:

A:5, B:9, C:12, D:13, E:16, F:45

Step 1 – Arrange by frequency

A(5), B(9), C(12), D(13), E(16), F(45)

Step 2 – Merge two smallest each time

1. **Merge** A(5) + B(9) = 14(AB)

AB(14), C(12), D(13), E(16), F(45)

Reorder by frequency:

C(12), D(13), AB(14), E(16), F(45)

2. Merge $C(12) + D(13) = 25(CD)$

Reorder:

$AB(14), E(16), CD(25), F(45)$

3. Merge $AB(14) + E(16) = 30(ABE)$

$CD(25), ABE(30), F(45)$

4. Merge $CD(25) + ABE(30) = 55(CDABE)$

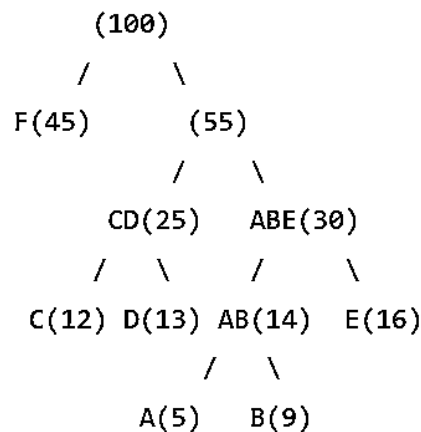
$F(45), CDABE(55)$

5. Merge $F(45) + CDABE(55) = 100(\text{root})$

Now we have **one root** — tree complete.

Step 3 – Assign 0s and 1s

Convention: **Left child = 0, Right child = 1**



Assign bits from root:

1. From **root**:

- Left branch → 0
- Right branch → 1

2. For $F(45)$:

- It's on the left of root →
code = 0

3. For (55) :

- It's on the right of root →
prefix 1

4. Inside (55) :

- Left child (CD) → 10

- Right child (ABE) → 11

5. Inside (CD) :

- Left child C → 100
- Right child D → 101

6. Inside (ABE) :

- Left child AB → 110
- Right child E → 111

7. Inside (AB) :

- Left child A → 1100
- Right child B → 1101

Step 4 – Final (Codes: A:5, B:9, C:12, D:13, E:16, F:45)

Symbol	Frequency	Code	Code length (bits)
F	45	0	1
C	12	100	3
D	13	101	3
E	16	111	3
A	5	1100	4
B	9	1101	4

Average bits per symbol =

$$(5 \times 4) + (9 \times 4) + (12 \times 3) + (13 \times 3) + (16 \times 3) + (45 \times 1) = 224 \text{ bits for 100 symbols}$$

$$\text{Average} = 224/100 = 2.24 \text{ bits/symbol}$$

$$\frac{\text{total bits}}{\text{total symbols}} = \frac{224}{100} = 2.24 \text{ bits/symbol}$$

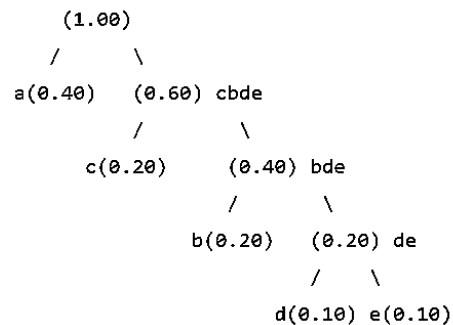
Examples 3:

Symbols: {a:0.4, b:0.2, c:0.2, d:0.1, e:0.1}

Start sorted by probability:

a(0.40), b(0.20), c(0.20), d(0.10), e(0.10)

Tree diagram (with left=0, right=1)



Final codes (from this tree):

Symbol	Probability	Code	Length (bits)
a	0.40	0	1
c	0.20	10	2
b	0.20	110	3
d	0.10	1110	4
e	0.10	1111	4

Average bits per symbol (expected length):

Compute $L = \sum p_i \cdot l_i$:

$$L = 0.4(1) + 0.2(2) + 0.2(3) + 0.1(4) + 0.1(4) = 0.4 + 0.4 + 0.6 + 0.4 + 0.4 = 2.2 \text{ bits/symbol}$$

So, on average each symbol requires **2.2 bits** with this Huffman coding.

Entropy:

$$\text{Entropy } H = - \sum p_i \log_2 p_i \approx 2.122 \text{ bits/symbol}$$

Huffman average = 2.20 bits/symbol — that's close to entropy

Questions For Solve:

1. A source emits symbols **{A, B, C, D}** with probabilities {0.4, 0.3, 0.2, 0.1}.
 - Construct the Huffman code, find the **average code length, entropy, efficiency, and redundancy**.
2. The probabilities of symbols are: {0.5, 0.25, 0.125, 0.125}.
 - Derive the Huffman code, compute the **entropy** and **average length**, and determine **if the code is optimal**.
3. A source produces the letters {M, N, O, P, Q} with probabilities {0.35, 0.25, 0.2, 0.1, 0.1}.
 - Construct the Huffman coding table and calculate:
 - (a) **Average length**
 - (b) **Entropy**
 - (c) **Efficiency**
4. Given probabilities: {0.4, 0.3, 0.2, 0.05, 0.05},
 - Find Huffman codes, **L_{avg}**, **Entropy H(X)**, **Efficiency η** , and **Redundancy R**.
5. The character set {X, Y, Z} has probabilities {0.6, 0.25, 0.15}.
 - Construct Huffman coding and compare the **actual code length** with **Shannon's minimum bound**.
6. A source has symbols with probabilities {0.33, 0.33, 0.17, 0.17}.
 - Find Huffman codes and verify **if efficiency can be 100%** for this case.
7. For symbols {A, B, C, D, E, F} with probabilities {0.05, 0.09, 0.12, 0.13, 0.16, 0.45},

- Construct Huffman codes, compute $H(X)$, L_{avg} , η , R , and verify Kraft's inequality.
8. A file has characters occurring with the following frequencies:
 {A: 15, B: 7, C: 6, D: 6, E: 5}.
- Determine Huffman codes, **average length**, **entropy**, **efficiency**.
9. Consider a biased coin where $P(\text{Head}) = 0.8$ and $P(\text{Tail}) = 0.2$.
- Encode with Huffman coding and calculate **redundancy**.
10. A source generates the digits {0, 1, 2, 3, 4} with probabilities {0.25, 0.25, 0.2, 0.15, 0.15}.
- Find Huffman code, L_{avg} , **Entropy**, η , and R .
 - Compare with a **fixed-length code**.